

Introduction

This is the exercise sheet for Lecture 1. During the lecture you saw a live demo of a small **Student/Programme** database. In this session you will:

- warm up, and then get more practice, with simple queries and inserts against that **Student/Programme** database,
- spot and fix the mistakes in a flawed **CREATE TABLE** statement,
- identify primary and foreign key candidates in a brand-new mini-domain, and
- design, and then implement in PostgreSQL, a small relational schema for a bookshop.

You can set up a local PostgreSQL instance using the **extras/** folder that accompanies this exercise sheet (see **How-to-run-PostgreSQL.md**). It will automatically load the **Student/Programme** demo data and a full solution to the bookshop schema.

1 Exercise 1: Warm-up – Querying Student and Programme

Recall the demo schema from the lecture:

Programme: {[programme_id: int, programme_name: string, duration_years: int]}

Student: {[student_id: int, first_name: string, last_name: string, email: string, birth_date: date, programme_id → Programme]}

Here, **Student.programme_id** records which degree programme a student is enrolled in: a genuine one-to-many relationship, since each student has exactly one programme, and each programme has many students.

Write SQL queries for the following. Test them against the demo database.

1. List the first name, last name, and birth date of all students.

Solution.

```
SELECT first_name, last_name, birth_date
FROM Student;
```

2. List the name and duration (in years) of all programmes longer than 2 years, ordered by duration descending.

Solution.

```
SELECT programme_name, duration_years
FROM Programme
WHERE duration_years > 2
ORDER BY duration_years DESC;
```

3. List the first and last name of every student enrolled in the programme with `programme_id = 1`, together with the name of that programme (a first taste of JOIN – covered properly in Lecture 5).

Solution.

```
SELECT s.first_name, s.last_name, p.programme_name
FROM Student s, Programme p
WHERE s.programme_id = p.programme_id
      AND s.programme_id = 1;
```

2 Exercise 2: More Practice – Querying and Modifying Student/Programme

A few more questions against the same demo database, this time also touching INSERT and a constraint you have already met: uniqueness.

1. Write an INSERT statement that adds a new student, Mette Poulsen (mette.poulsen@student.aau.dk, born 2003-05-20), enrolled in the “Computer Science” programme (programme_id = 2).

Solution.

```
INSERT INTO Student (first_name, last_name, email,
    birth_date, programme_id)
VALUES ('Mette', 'Poulsen', 'mette.poulsen@student.
    aau.dk', '2003-05-20', 2);
```

2. List the name and duration (in years) of every programme that is shorter than 3 years, ordered alphabetically by programme name.

Solution.

```
SELECT programme_name, duration_years
FROM Programme
WHERE duration_years < 3
ORDER BY programme_name;
```

3. Without running it, explain in one sentence why the following INSERT would be rejected by the database. Then run it yourself to check your answer.

```
INSERT INTO Student (first_name, last_name, email,
    birth_date, programme_id)
VALUES ('Test', 'Student', 'anna.jensen@student.aau
    .dk', '2005-01-01', 1);
```

Solution.

It violates the UNIQUE constraint on Student.email: the value 'anna.jensen@student.aau.dk' is already used by the existing row for Anna Jensen, and email was declared UNIQUE NOT NULL back in the live demo – so no two students may share the same email address, even though every other column value in the new row is otherwise perfectly valid.

3 Exercise 3: Spot the Problem – A Flawed CREATE TABLE

A classmate wrote the following two tables for tracking a company's employees and departments, but made several mistakes. Find as many problems as you can – in terms of the keys and integrity rules from the lecture – and rewrite the statements to fix them.

```
CREATE TABLE Employee (  
    emp_id      INTEGER,  
    first_name  VARCHAR(50) NOT NULL,  
    last_name   VARCHAR(50) NOT NULL,  
    salary      VARCHAR(20)  
);  
  
CREATE TABLE Department (  
    dept_id     INTEGER,  
    dept_name   VARCHAR(100),  
    emp_id      INTEGER,  
    FOREIGN KEY (emp_id) REFERENCES Employee(emp_id)  
);
```

Hint: think about which table the relationship “every employee belongs to exactly one department, and a department has many employees” should put the foreign key on, and check every column against the three integrity rules from the lecture.

Solution.

Problems in the original:

- *Employee.emp_id* has no **PRIMARY KEY** – nothing enforces entity integrity, so two employees could end up with the same id, or none at all.
- *Department.dept_id* has no **PRIMARY KEY** either, for the same reason.
- *Department.dept_name* is missing **NOT NULL** – a department without a name should not be allowed.
- *salary VARCHAR(20)* is the wrong data type – a salary is a number, so nothing stops someone inserting 'not telling', which breaks domain integrity.
- The foreign key is on the wrong table and points the wrong way: *Department.emp_id REFERENCES Employee(emp_id)* would mean each department links to at most one employee. The real relationship is the other way around – many employees belong to one department – so the foreign key belongs on *Employee*, referencing *Department*.

A corrected version:

```
CREATE TABLE Department (  
    dept_id      SERIAL PRIMARY KEY,  
    dept_name    VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE Employee (  
    emp_id       SERIAL PRIMARY KEY,  
    first_name   VARCHAR(50) NOT NULL,  
    last_name    VARCHAR(50) NOT NULL,  
    salary       DECIMAL(10,2) NOT NULL,  
    dept_id      INTEGER NOT NULL REFERENCES Department(  
        dept_id)  
);
```

4 Exercise 4: Identify the Keys – A Gym’s Members and Branches

No SQL for this one – just work from the plain-English description below.

A small gym chain wants a database for its members and branches (locations). Every member signs up at exactly one branch when they join, and every branch has many members. A member has a name, an email address, and a join date. A branch has a name and a city.

Answer the following, in words:

- What are the two tables (entities) here, and what are their columns?
- What is a sensible primary key for each table?
- Where do you need a foreign key, and which table/column does it reference?
- In one sentence, describe the cardinality of the relationship: how many branches can one member belong to, and how many members can one branch have?

Solution.

Branch: {[branch_id: int, branch_name: string, city: string]}

Member: {[member_id: int, name: string, email: string, join_date: date, branch_id → Branch]}

Branch.branch_id is the primary key of Branch; Member.member_id is the primary key of Member. Member.branch_id is a foreign key referencing Branch.branch_id. The relationship is one-to-many: each member belongs to exactly one branch, but a branch can have many members – exactly the same shape as Student and Programme from the lecture.

5 Exercise 5: Designing a Bookshop Schema

A small online bookshop wants to keep track of the books it sells, the authors who wrote them, and the orders customers place.

On paper, design a simple relational schema with 2–3 tables – **Book**, **Author**, and **Order** – that could support this. For each table:

- list its columns and a sensible data type for each;
- underline the primary key;
- mark any column that is a foreign key, and say which table/column it references;
- briefly say, in words, what the relationship between the tables is (e.g. “one author can write many books”).

You do not need full ER notation (with cardinalities and participation constraints) – that is introduced in Lecture 2. For now, “table + columns + which column references which” is enough. Discuss your design with a neighbour: did you make the same choices?

Hint: think about what happens if a book has more than one author. A simple starting design may assume one author per book – you can note that as a simplification.

Solution.

One valid design (assuming, for simplicity, exactly one author per book – a full solution allowing multiple authors per book needs an extra “many-to-many” table, which is discussed once relationships of that kind are covered in Lecture 2):

Author: {[author_id: int, name: string, country: string]}

Book: {[isbn: string, title: string, price: decimal, author_id → Author]}

Order: {[order_id: int, isbn → Book, customer_name: string, quantity: int, order_date: date]}

In words:

- *Author.author_id is the primary key of Author.*
- *Book.isbn is the primary key of Book; Book.author_id is a foreign key referencing Author.author_id – one author can write many books, but here each book has exactly one author.*
- *Order.order_id is the primary key of Order; Order.isbn is a foreign key referencing Book.isbn – one book can appear in many orders, but each order (row) is for one book.*

6 Exercise 6: Implementing the Bookshop Schema

Create a PostgreSQL database locally (see `How-to-run-PostgreSQL.md` in the `extras/` folder for a ready-to-use Docker setup).

1. Write `CREATE TABLE` statements for the `Author`, `Book`, and `Order` tables from Exercise 5. Use explicit constraints: primary keys, foreign keys, `NOT NULL` where appropriate, and reasonable data types.
2. Insert at least 5 rows into each table. Pay attention to the order in which you insert rows – what does the foreign key constraint require?
3. Write one `SELECT` query that lists, for every order, the customer name, the book title, and the author's name (i.e. a query that joins all three tables).

Solution.

Tables:

```
CREATE TABLE Author (  
    author_id    SERIAL PRIMARY KEY,  
    name         VARCHAR(100) NOT NULL,  
    country      VARCHAR(60)  
);
```

```
CREATE TABLE Book (  
    isbn         VARCHAR(20) PRIMARY KEY,  
    title        VARCHAR(200) NOT NULL,  
    price        DECIMAL(6,2) NOT NULL CHECK (price >=  
        0),  
    author_id    INTEGER NOT NULL,  
    FOREIGN KEY (author_id) REFERENCES Author (  
        author_id)  
);
```

```
CREATE TABLE "Order" (  
    order_id     SERIAL PRIMARY KEY,  
    isbn         VARCHAR(20) NOT NULL,  
    customer_name VARCHAR(100) NOT NULL,  
    quantity     INTEGER NOT NULL CHECK (quantity >  
        0),  
    order_date   DATE NOT NULL DEFAULT CURRENT_DATE,  
    FOREIGN KEY (isbn) REFERENCES Book (isbn)  
);
```

Note: ORDER is a reserved SQL keyword, so the table name must be quoted as "Order" wherever it is used.

Data (at least 5 rows per table):

```
INSERT INTO Author (name, country) VALUES
('J.K. Rowling', 'United Kingdom'),
('George Orwell', 'United Kingdom'),
('Haruki Murakami', 'Japan'),
('Isabel Allende', 'Chile'),
('Yuval Noah Harari', 'Israel');
```

```
INSERT INTO Book (isbn, title, price, author_id) VALUES
('978-0-7475-3269-9', 'Harry Potter and the Philosopher
    's Stone', 12.99, 1),
('978-0-452-28423-4', '1984', 9.99, 2),
('978-0-307-59313-3', 'Norwegian Wood', 11.50, 3),
('978-0-06-088328-7', 'The House of the Spirits',
    13.25, 4),
('978-0-06-231609-7', 'Sapiens', 15.00, 5);
```

```
INSERT INTO "Order" (isbn, customer_name, quantity,
    order_date) VALUES
('978-0-7475-3269-9', 'Anna Nielsen', 2, '2026-01-10'
),
('978-0-452-28423-4', 'Peter Jensen', 1, '2026-01-11'
),
('978-0-307-59313-3', 'Mette Sorensen', 3, '2026-01-12'
),
('978-0-06-088328-7', 'Anna Nielsen', 1, '2026-01-15'
),
('978-0-06-231609-7', 'Lars Christensen', 2, '
    2026-01-16');
```

Join query:

```
SELECT o.customer_name, b.title, a.name AS author_name
FROM "Order" o
JOIN Book b ON o.isbn = b.isbn
JOIN Author a ON b.author_id = a.author_id;
```

AI Disclosure

Claude, a large language model developed by Anthropic, was used to assist with structuring and grammar checking of this material. All generated text was critically reviewed, edited, and verified by the author.